Labo 17 — Praktijklabo: packagen, opleveren, terugdraaien

Doel: Helm installeren, ontdekken dat k3s het al gebruikt, de volledige stack (web, api, db) in een chart packagen, ze installeren, bijwerken, terugdraaien, hetzelfde renderen met Kustomize, en daarna de cluster laten pullen vanuit de registry van labo 10, per tag en daarna per digest. En, om de reeks af te sluiten, alles opruimen.

Vereisten — Labo 16 afgerond: k3s; `localhost/api:1.0` en `localhost/api:1.1` geïmporteerd in containerd (labo 12); de image `registry:2` en `api:1.1` in Podman (labo's 10 en 12).

Meegeleverde bestanden (`files/`) - `stack/` — de Helm chart: `Chart.yaml` , `values.yaml` , `values-prod.yaml` , `templates/` , `files/` . - `kustomize/base/` en `kustomize/overlays/prod/` — dezelfde API als Kustomize base + overlay.

Stap 1 — Helm installeren, en het al aan het werk zien

```
mkdir -p ~/labo-docker/17 && cd ~/labo-docker/17 && cp -r <labo-pad>/files/* .
curl -fsSL https://get.helm.sh/helm-v4.3.1-linux-amd64.tar.gz | tar xz -C /tmp
sudo install /tmp/linux-amd64/helm /usr/local/bin/helm
helm version --short
helm list -A
```

Observeer `v4.3.1+gc0d08d0` , en daarna `Error: kubernetes cluster unreachable: Get "http://localhost:8080/version": dial tcp [::1]:8080: connect: connection refused` .

```
export KUBECONFIG=/etc/rancher/k3s/k3s.yaml
helm list -A
kubectl get helmcharts -n kube-system
```

Observeer twee releases in `kube-system` : `traefik` en `traefik-crd` , chart `traefik-34.2.1+up34.2.0` , `deployed` sinds de installatie van k3s; en de `HelmChart` -objecten die ze gemaakt hebben.

Uitleg. Helm is een gewone client: zonder `KUBECONFIG` weet het niet waar de cluster is. En de Ingress controller die je sinds labo 13 gebruikt, is een Helm chart die k3s geïnstalleerd heeft.

> **→ WINDOWS / WSL**
> Voeg `export KUBECONFIG=/etc/rancher/k3s/k3s.yaml` toe aan je `~/.bashrc` : Helm, maar ook k9s of Lens aan de Windows-kant, hebben het nodig. De `kubectl` van k3s vindt het bestand zelf.

Stap 2 — De chart lezen, renderen, installeren

```
ls -R stack | head -20
helm lint stack
helm template stack ./stack | grep -E '^kind:|^  name:' | paste - - | head -12
helm template stack ./stack --set api.replicas=5 | grep -m1 'replicas:'
```

Observeer de vijf mappen en bestanden van de chart; `1 chart(s) linted, 0 chart(s) failed` (een `[INFO] icon is recommended` is onschuldig); elf gerenderde objecten (`Secret db-secret`, drie `ConfigMap`s, drie `Service`s, twee `Deployment`s, het `StatefulSet db`, de `Ingress web`); en `replicas: 5` zodra je een waarde overschrijft.

```
helm install stack ./stack -n lab17 --create-namespace
kubectl config set-context --current --namespace=lab17
sleep 25 ; kubectl get pods
helm list
curl -s localhost/api/ ; echo
```

Observeer (een waarschuwing `SessionAffinity is ignored for headless services` is zonder gevolg) `STATUS: deployed`, `REVISION: 1`, daarna de tekst van `NOTES.txt` met de gerenderde waarden; vier Pods `Running` (`api` ×2, `db-0`, `web`); `stack lab17 1 … deployed stack-0.1.0 1.0`; en, via de Ingress en de nginx-proxy, `{"message":"bonjour depuis Helm","profile":"dev"}`.

Uitleg. Eén commando heeft elf objecten gemaakt uit een map en een waardenbestand. `helm template` toont ze, `helm lint` controleert ze, `helm install` past ze toe en nummert de revision.

Stap 3 — Bijwerken, raadplegen, terugdraaien

```
helm upgrade stack ./stack --reuse-values --set api.message="bonjour depuis la revision 2" --
set api.replicas=3
kubectl rollout status deployment/api --timeout=90s
curl -s localhost/api/ ; echo
helm history stack
helm get values stack
```

Observeer `Release "stack" has been upgraded. Happy Helming!`; een rolling update van de API; de nieuwe boodschap in het antwoord; twee revisions (`1 superseded`, `2 deployed … Upgrade complete`); en onder `USER-SUPPLIED VALUES` precies je twee `--set`'s.

```
helm rollback stack 1
kubectl rollout status deployment/api --timeout=90s ; sleep 5
curl -s localhost/api/ ; echo
kubectl get pods -l app=api
helm history stack
```

Observeer `Rollback was a success!`; de oorspronkelijke boodschap is terug; twee `api`-Pods (geen drie meer); en een **derde** revision `deployed`, beschreven als `Rollback to 1`: de twee vorige zijn `superseded`.

Uitleg. Een ConfigMap-waarde veranderen heeft de API herdeployd dankzij de annotatie `checksum/config`. De rollback wist de geschiedenis niet: hij verlengt ze met een revision die identiek is aan de eerste.

Stap 4 — Een waardenbestand per omgeving

```
cat stack/values-prod.yaml
helm template stack ./stack -f stack/values-prod.yaml | grep -E 'replicas:|APP_PROFILE|memory'
helm upgrade stack ./stack -f stack/values-prod.yaml --dry-run 2>&1 | grep -c 'kind:'
```

Observeer een bestand dat **alleen de verschillen** met `values.yaml` bevat; in de rendering `APP_PROFILE: "prod"`, `replicas: 3` voor de API, `memory: 512Mi` / `128Mi`, `replicas: 2` voor het web; en een `--dry-run` dat elf objecten rendert zonder iets aan te raken.

Uitleg. De templates veranderen nooit van omgeving tot omgeving; alleen het waardenbestand dat je met `-f` meegeeft verandert. Het is de `compose.prod.yaml` van labo 09, voor een cluster.

Stap 5 — Hetzelfde met Kustomize

```
cat kustomize/overlays/prod/kustomization.yaml
kubectl kustomize kustomize/overlays/prod | grep -E
'^kind:|namespace:|replicas:|image:|env:|memory:'
kubectl apply -k kustomize/overlays/prod
sleep 20 ; kubectl get pods -n lab17-prod -o custom-
columns=NAME:.metadata.name,IMAGE:.spec.containers[0].image,ENV:.metadata.labels.env
kubectl delete -k kustomize/overlays/prod
```

Observeer een overlay die naar `../../base` verwijst en `namespace`, `labels`, `replicas`, `images`, `patches` declareert; een rendering waarin de ongewijzigde base een Deployment met `replicas: 3`, image `localhost/api:1.1`, label `env: prod` en gepatchte resources geworden is, in `lab17-prod`; drie conforme Pods; en daarna het verwijderen van de namespace en zijn inhoud.

Uitleg. Geen enkel template: zuivere YAML, en een overlay die zegt wat er verandert. `kubectl` kan renderen (`kustomize`) en toepassen (`-k`) zonder iets te installeren.

Stap 6 — Pullen vanuit de registry, per tag en daarna per digest

Aan de Podman-kant (labo 10): start de registry opnieuw en push `api:1.1`, met notering van de digest.

```
podman run -d --name registry -p 5000:5000 docker.io/library/registry:2 ; sleep 2
podman tag api:1.1 localhost:5000/api:1.1
podman push --tls-verify=false --digestfile digest.txt localhost:5000/api:1.1 2>&1 | tail -1
cat digest.txt ; echo
podman image inspect api:1.1 --format '{{.Digest}}'
```

Observeer `Writing manifest to image destination`, een digest `sha256:b82020cb…` in `digest.txt`, en een **andere** digest (`sha256:e1b2ba1e…`) voor de lokale image: de push heeft de lagen gehercomprimeerd; die van de registry is de enige die telt.

```
curl -s localhost:5000/v2/api/tags/list ; echo
helm upgrade stack ./stack --reuse-values --set api.image.repository=localhost:5000/api --set
api.image.tag=1.1
kubectl rollout status deployment/api --timeout=90s
kubectl get events --field-selector reason=Pulled | grep -m1 5000 | cut -c1-150
kubectl get pods -l app=api -o jsonpath='{.items[0].status.containerStatuses[0].imageID}' ;
echo
```

Observeer `{"name":"api","tags":["1.1"]}`; een rolling update; `Successfully pulled image "localhost:5000/api:1.1" in 92ms` (jouw waarden verschillen) — k3s heeft gepulld vanuit de registry, over HTTP, zonder `registries.yaml`; en `imageID` = `localhost:5000/api@sha256:b82020cb…`: dezelfde digest als in `digest.txt`.

```
DIGEST=$(kubectl get pods -l app=api -o
jsonpath='{.items[0].status.containerStatuses[0].imageID}' | cut -d@ -f2) ; echo $DIGEST
helm upgrade stack ./stack --reuse-values --set api.image.digest=$DIGEST
kubectl rollout status deployment/api --timeout=90s
kubectl get deployment api -o jsonpath='{.spec.template.spec.containers[0].image}' ; echo
helm history stack | cut -c1-110
```

Observeer een Deployment waarvan de image nu `localhost:5000/api@sha256:b82020cb…` is: geen tag meer, een vingerafdruk; en vijf revisions in de geschiedenis.

Uitleg. De tag `1.1` zou morgen herschreven kunnen worden; de digest nooit. De CI pusht met een onveranderlijke tag en noteert de digest; de productie pint de digest vast.

> **→ WINDOWS / WSL**
> Op WSL zitten Podman en k3s op dezelfde machine: `localhost:5000` is dezelfde registry voor beide, en containerd aanvaardt `localhost` onversleuteld. Voor een registry op een andere machine of met inloggegevens: `/etc/rancher/k3s/registries.yaml`, en daarna `sudo systemctl restart k3s`.

Opruimen — en einde van de reeks

```
helm uninstall stack ; sleep 15
kubectl get all,pvc
kubectl delete namespace lab17 ; kubectl config set-context --current --namespace=default
helm list -A
podman rm -f -t 0 -v registry ; podman rmi localhost:5000/api:1.1
```

Observeer na `uninstall` geen enkele Pod, Service of Deployment meer… maar de PVC `data-db-0` nog altijd `Bound`: Helm verwijdert niet wat het StatefulSet gemaakt heeft. Het verwijderen van de namespace neemt ze mee. `helm list -A` toont alleen nog Traefik.

Om **heel** k3s van je WSL te verwijderen, als je klaar bent (onomkeerbaar, verwijdert ook de `local-path`-volumes):

```
sudo /usr/local/bin/k3s-uninstall.sh
```

De Podman-images (`api:1.0`, `api:1.1`, `web:1.0`, `postgres`, `nginx`) en het volume `stack_pgdata` van labo 09 blijven van jou: `podman images`, `podman volume ls`, en labo 10 om op

te ruimen.

Wat je nu moet kunnen beweren

- `helm` zonder `KUBECONFIG` zoekt `localhost:8080`; ermee toont `helm list -A` de Traefik die k3s geïnstalleerd heeft.
- Chart `stack/`: `lint`, `template` (elf objecten), `install` (revision 1, `NOTES.txt`), `curl localhost/api/`.
- `upgrade --set` → revision 2 en nieuwe boodschap; `rollback 1` → revision 3 `Rollback to 1`.
- `-f values-prod.yaml`: alleen de verschillen; `--dry-run` rendert zonder toe te passen.
- Kustomize: `kubectl kustomize` toont, `apply -k` past toe, overlay = namespace + replicas + images + patches + labels.
- Registry: `--digestfile` = digest van de registry = `imageID` op de node; `image@sha256:` in het Deployment.
- `helm uninstall` bewaart de PVC; `k3s-uninstall.sh` verwijdert alles.